

业务型 Agent 项目推荐报告

业务型 Agent 项目选择报告 · 源码验证版 · 简历改造路线

生成日期	2026-07-07
推荐模式	mixed
用户画像	按默认画像处理，适合早期后端、Java 后端、智能体应用工程方向候选人。
筛选口径	业务型 Agent 必须有业务数据、状态流转、持久化、工具调用、评测或用户价值；传统后端默认排除 IoT、硬件接入、设备管理和工业控制类项目。
可信度边界	负责功能来自 pull_github_repos.py 拉取到本地后的源码证据；README、候选池和网页信息只用于前置筛选，不支撑简历负责功能。未完成二次改造的内容统一放入“建议简历功能点（完成对应改造后可写）”。

结论先行

- 推荐组合：xerrors/Yuxi + newbee-ltd/newbee-mall。
- 模式解释：mixed 同时补足“智能体差异化”和“后端基本功”。Yuxi 适合包装为企业知识库 Agent / 多智能体平台，newbee-mall 适合包装为电商交易后端。
- 定位分工：Yuxi 负责展示 Agent 工程深度，包括 AgentRun、子智能体、RAG 工具、Redis 事件流、评测链路；newbee-mall 负责展示订单、库存、支付状态、后台履约、权限拦截和 MyBatis 数据访问。
- 选择原因：两个项目都能从本地源码提取可追问的服务层、仓储层、状态流转和异常边界，且技术卖点互补，不会都停留在“接入大模型”或“普通 CRUD”。

项目候选池

项目	链接	分桶	推荐模式匹配	定位	推荐度	取舍理由
xerrors/Yuxi	https://github.com/xerrors/Yuxi	企业知识库 Agent	高	业务型 Agent	5/5	多租户知识库、AgentRun、子智能体、MCP/工具、评测、Redis 事件流完整，适合智能体方向简历。
newbee-ltd/newbee-mall	https://github.com/newbee-ltd/newbee-mall	电商交易	高	传统软件后端	4/5	Java/Spring Boot 电商链路完整，订单、库存、购物车、支付状态、后台履约容易被面试追问。
vm0-ai/vm0	https://github.com/vm0-ai/vm0	业务型 Agent	中	Agent 工作流平台	备选	工具编排和企业自动化场景强，但偏 TypeScript / DevOps 产品，默认画像下不如 Yuxi 贴近知识库 Agent。
Peppermint-Lab/peppermint	https://github.com/Peppermint-Lab/peppermint	工单客服	中	传统后端业务系统	备选	工单业务清晰，但技术栈偏 Node/Next/Prisma；若目标 Java 后端，不如 newbee-mall 匹配。

多样性说明

- 搜索覆盖方向：业务型 Agent、多智能体、企业知识库 Agent、数据分析 Agent、电商交易、工单客服、CRM/ERP。

- 分桶策略：最终保留一个 Agent 项目和一个纯软件后端业务项目；后端没有选择 IoT、嵌入式、设备管理或工业控制类项目。
- 过热降权：没有直接选择通用 Agent 框架或浏览器插件；newbee-mall 虽是热门 Java 电商项目，但源码中订单状态、库存扣减和后台履约链路完整，保留为扎实后端项目。
- 本次组合价值：Yuxi 负责“新奇差异化”，newbee-mall 负责“后端可追问深度”。

可替换项目

- 如果不想做企业知识库 Agent，可以把 Yuxi 换成 vm0-ai/vm0，方向转为企业自动化 Agent / 工具编排平台。
- 如果不想做 Java 电商，可以把 newbee-mall 换成 Peppermint-Lab/peppermint，方向转为工单客服 / 内部协作系统。

推荐项目 1：Yuxi

- 项目定位：业务型 Agent / 企业知识库智能体平台。
- 链接：<https://github.com/xerrors/Yuxi>
- 适合人群：想投智能体应用工程、大模型平台、RAG 后端、Agent 后端的候选人。
- 为什么适合写简历：它不是单次 LLM 调用包装，而是包含 FastAPI 服务、AgentRun 生命周期、Redis 事件流、后台 worker、子智能体、知识库工具、知识图谱数据模型和评测数据集的完整业务系统。
- 已有能力：多租户知识库、Agent 执行队列、SSE 事件流、运行取消、子智能体任务工具、MCP 动态工具、知识库检索工具、评测数据集与指标聚合。
- 代码验证摘要：repo-source-manifest.json 中 xerrors/Yuxi 状态为 cloned，本地目录 /workspace/.repo-source-cache/xerrors_Yuxi，commit a51eb26d529f9e2e4c6295f8f4b82a81c70e65d7。已阅读 backend/server/main.py、backend/package/yuxi/services/agent_run_service.py、backend/package/yuxi/services/run_queue_service.py、backend/package/yuxi/services/run_worker.py、backend/package/yuxi/agents/middlewares/subagent_task.py、backend/package/yuxi/services/subagent_run_service.py、backend/package/yuxi/agents/toolkits/kbs/tools.py、backend/package/yuxi/agents/middlewares/dynamic_tool.py、backend/package/yuxi/storage/postgres/models_knowledge.py、backend/package/yuxi/repositories/evaluation_repository.py、backend/package/yuxi/knowledge/eval/evaluator.py。
- 建议二次改造：补充面向企业知识库的“任务审批 + 引用溯源 + 评测看板 + 工具超时降级”闭环，把项目从平台源码学习升级为可演示的业务 Agent 系统。

简历写法

项目简介

本项目基于 Yuxi 搭建企业知识库智能体平台，面向企业文档问答、知识图谱检索和多智能体任务执行场景，将知识库管理、AgentRun 后台执行、子智能体协作、工具调用、运行事件追踪与评测数据集串联起来，形成可观测、可恢复、可评估的业务型 Agent 后端系统。

负责功能 / 技术难点

- AgentRun 生命周期编排：把普通对话、外部调用、评测运行和子智能体任务统一落到 AgentRun 记录，使用 request_id 做幂等校验，并在创建后投递 ARQ worker，避免不同入口各自维护执行路径。
- 运行事件流设计：用 Redis Stream 保存 run 事件，SSE 接口按游标读取增量消息，同时保留心跳、事件压缩和终态补发逻辑，解决长任务执行中前端断线重连与进度恢复问题。

- 子智能体任务隔离：在父 run 作用域内创建 SubagentThread 关系，提供 subagent_start/status/events/cancel/await 工具，禁止子智能体继续创建子智能体，并通过 busy 校验防止同一子线程并发写入。
- 知识库工具访问控制：list_kbs/query_kb/open_kb_document 根据 runtime context 解析当前用户可见知识库，只允许查询会话启用的 kb_id，减少 Agent 工具越权读取知识库的风险。
- 动态 MCP 工具治理：启动时预加载 MCP 工具，运行时按 Agent 配置筛选基础工具和 MCP 工具，避免模型在一次调用中看到全部工具，降低误调用和工具暴露面。
- RAG 评测闭环：用 EvaluationDataset/EvaluationRun/EvaluationRunItem 记录评测数据和运行结果，评测器对召回 chunk 与标准答案计算检索和答案指标，为知识库问答效果提供可量化追踪。

建议简历功能点（完成对应改造后可写）

- 增加 Agent 工具调用超时和降级策略：为知识库检索、MCP 调用和子智能体任务分别设置超时、重试与失败摘要，避免长任务阻塞主 run。
- 构建企业知识库评测看板：把评测运行的 retrieval_metrics、答案正确率和 overall score 汇总到可视化接口，支持按知识库版本对比 RAG 效果。
- 改造引用溯源链路：在 query_kb 返回结果中统一携带 file_id、chunk 位置和图谱实体关系，回答生成后可追溯到原始文档窗口。
- 补充人工审批节点：对高风险工具调用加入 human approval 状态，审批结果写入 AgentRun 事件流并支持取消或继续执行。
- 增强多租户权限审计：记录知识库查询、文档打开、MCP 工具调用和子智能体启动日志，支持按用户、知识库、工具维度追踪敏感操作。

可改造方向

- 把“知识库问答”包装成“企业知识运营助手”：支持上传制度文档、生成思维导图、检索引用、生成回答、评测准确率。
- 把“子智能体”包装成“多角色任务协作”：主 Agent 拆分需求，检索 Agent 查资料，报告 Agent 输出结构化结论。
- 把“运行事件流”包装成“可观测 Agent 后台任务中心”：展示 running、cancel_requested、completed、failed 等状态和最近消息。

推荐项目 2：newbee-mall

- 项目定位：扎实项目 / Java Spring Boot 电商交易后端。
- 链接：<https://github.com/newbee-ltd/newbee-mall>
- 适合人群：想投 Java 后端、后端实习、业务系统开发方向的候选人。
- 为什么适合写简历：源码覆盖前台商城、购物车、订单结算、支付状态、个人订单、后台订单履约、商品上下架、登录拦截和 MyBatis 数据访问，比单纯管理后台 CRUD 更容易讲清交易链路。
- 已有能力：Spring Boot + Thymeleaf + MyBatis + MySQL，包含商品检索、购物车限制、订单创建、库存扣减/恢复、支付成功、订单取消/完成、后台配货/出库/关闭、用户登录拦截。
- 代码验证摘要：repo-source-manifest.json 中 newbee-ltd/newbee-mall 状态为 cloned，本地目录 /workspace/.repo-source-cache/newbee-ltd_newbee-mall，commit a069069b07027613bf0e7f571736be86f431faee。已阅读 pom.xml、
src/main/java/ltd/newbee/mall/service/impl/NewBeeMallOrderServiceImpl.java、
src/main/java/ltd/newbee/mall/service/impl/NewBeeMallShoppingCartServiceImpl.java、
src/main/java/ltd/newbee/mall/controller/mall/OrderController.java、
src/main/java/ltd/newbee/mall/controller/admin/NewBeeMallOrderController.java、
src/main/java/ltd/newbee/mall/common/NewBeeMallOrderStatusEnum.java、
src/main/java/ltd/newbee/mall/common/Constants.java、
src/main/java/ltd/newbee/mall/config/NeeBeeMallWebMvcConfigurer.java、
src/main/resources/mapper/NewBeeMallOrderMapper.xml、
src/main/resources/mapper/NewBeeMallGoodsMapper.xml、
src/main/resources/mapper/NewBeeMallShoppingCartItemMapper.xml、
src/main/resources/newbee_mall_schema.sql。
- 建议二次改造：补充 Redis 缓存、MQ 延迟关单、支付回调幂等、库存预占日志和订单状态机表，把传统商城提升为更有后端追问深度的交易系统。

简历写法

项目简介

本项目基于 newbee-mall 搭建 Java 电商交易后端，模拟用户浏览商品、加入购物车、提交订单、选择支付、后台配货出库和用户确认收货的完整交易流程。系统通过 Spring Boot、MyBatis 和 MySQL 管理商品、购物车、订单、库存和登录会话，适合作为后端业务链路项目进行二次改造。

负责功能 / 技术难点

- 订单创建链路：提交订单时批量读取购物车商品、校验商品上架状态和库存数量，事务内删除购物车、扣减库存、生成订单号并写入订单项快照，避免订单与购物车状态不一致。

- 库存扣减边界：MyBatis 批量更新库存时增加 `stock_num >= goodsCount` 和上架状态条件，库存不足直接中断订单创建，防止超卖商品进入待支付订单。
- 后台履约状态流转：管理端把订单分为已支付、配货完成、出库成功、交易成功和关闭状态，配货、出库、关闭操作在服务层校验当前状态并返回异常订单号，减少后台误操作。
- 用户侧订单权限校验：订单详情、支付页面、取消订单和确认收货都校验订单归属用户，结合登录拦截器保护购物车、订单、个人中心等路径，避免越权查看或修改他人订单。
- 购物车容量治理：新增购物车时限制单商品数量和购物车总条目数，重复商品走更新逻辑，删除使用软删除字段，保证用户购物车数据不会无限膨胀。
- 支付状态更新：支付成功接口只允许待支付订单变更为已支付，写入 `pay_type`、`pay_status`、`pay_time` 和更新时间，为后续配货和出库动作提供明确状态前置条件。

建议简历功能点（完成对应改造后可写）

- 设计订单状态机：把待支付、已支付、配货、出库、完成、取消等动作统一抽象成状态迁移表，增加非法迁移拦截和状态变更审计日志。
- 接入 Redis + 本地缓存优化商品查询：对首页推荐、搜索热词和商品详情做缓存预热，并处理商品下架后的缓存失效。
- 增加支付回调幂等表：使用订单号 + 第三方流水号唯一约束处理重复回调，避免同一订单被多次支付成功更新。
- 实现延迟关单：订单创建后发送延迟消息或定时扫描，超时未支付自动关闭订单并恢复库存，补齐库存回滚链路。
- 改造库存预占模型：订单创建时写库存流水，支付超时或后台关闭时按订单项恢复库存，支持后续对账和补偿任务。
- 建立订单操作日志：记录用户取消、支付成功、后台配货、出库、关闭等动作，支持后台排查异常状态流转。

可改造方向

- 优先补“支付回调幂等 + 延迟关单 + 库存流水”，这是电商后端最容易被追问的交易一致性链路。
- 再做“商品缓存 + 搜索索引”，把项目从普通商城提升到可讲高并发读和索引一致性的后端项目。
- 最后补“订单状态机 + 操作日志”，让业务状态流转更规范，也更适合简历和面试复盘。

最终建议

- 如果只能先做一个项目：优先做 newbee-mall，Java 后端通用性更强，容易覆盖实习面试中的订单、库存、权限、事务问题。
- 如果想差异化更强：再做 Yuxi，把知识库 Agent、子智能体、工具调用、Redis 事件流和评测闭环讲清楚。

- 简历组合建议：把 newbee-mall 放在“后端项目”位置，Yuxi 放在“智能体平台 / 大模型应用后端”位置；两者不要都写成全栈项目，重点分别放在交易链路和 Agent 工程。